



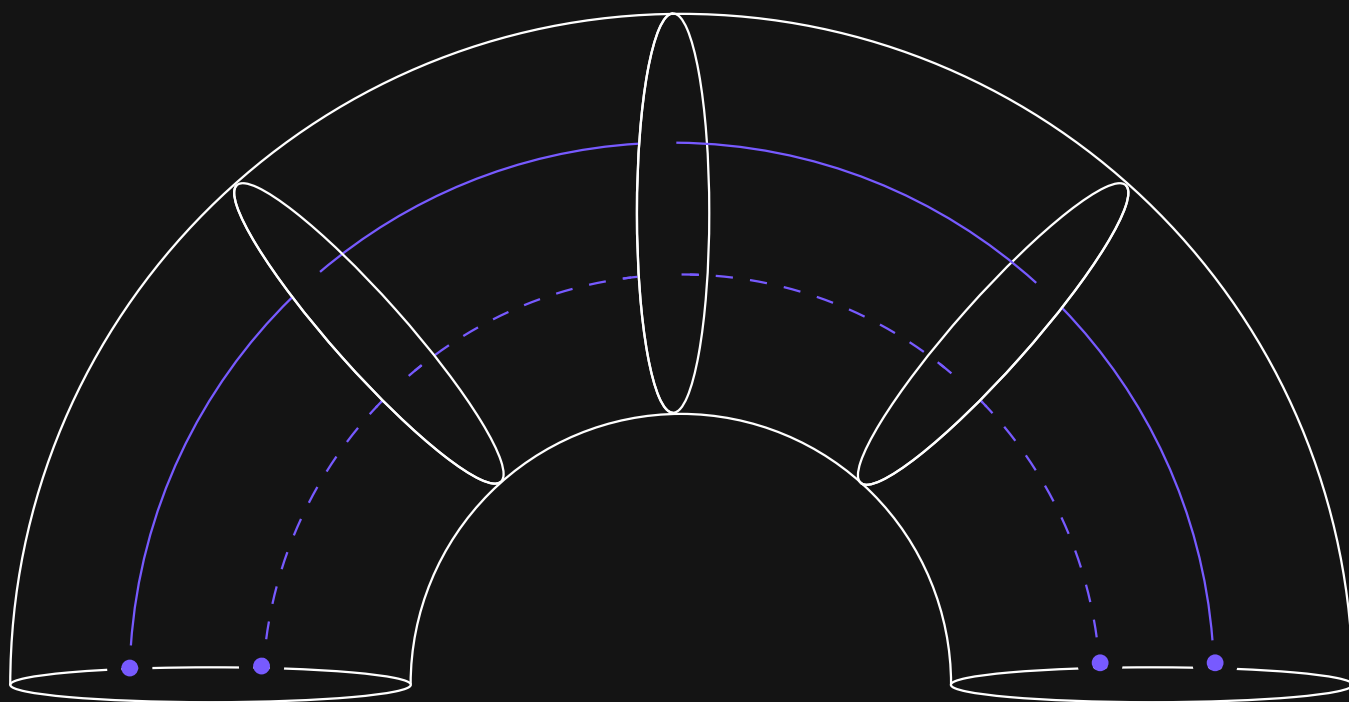
Машинное обучение

Лонгрид

Глубокое обучение

04

# Задачи CV: семантическая сегментация



# Содержание

|                                                         |    |
|---------------------------------------------------------|----|
| 1. От классификации к сегментации .....                 | 3  |
| 2. Постановка задачи семантической сегментации .....    | 3  |
| 3. Энергетические модели и случайные поля .....         | 4  |
| 4. Зачем нужны парные потенциалы: немного истории ..... | 6  |
| 5. Обучение: градиент лог-правдоподобия .....           | 7  |
| 6. Сэмплирование по Гиббсу .....                        | 8  |
| 7. Этап теста .....                                     | 10 |
| 8. Метрика качества: коэффициент Жаккара .....          | 10 |
| 9. Частный случай: только унарные потенциалы .....      | 11 |
| 10. Данные для сегментации .....                        | 12 |
| 11. Архитектуры: SegNet, U-Net, LinkNet .....           | 13 |
| 12. Пирамидальный пулинг и свёртки с дырками .....      | 19 |
| 13. Что дальше .....                                    | 21 |

# 1. От классификации к сегментации

В прошлой лекции были свёрточные нейронные сети: основные архитектуры и задача классификации изображений — дана картинка, нужно найти на ней кота или собаку. Теперь хотелось бы решать с картинками и свёрточными архитектурами более хитрые постановки. Эта лекция — про задачу так называемой *семантической сегментации* изображения: на вход подаётся картинка, а на выходе для каждого пикселя нужно сказать, к какому сегменту из ограниченного набора сегментов этот пиксель принадлежит. Например, городская сцена: вот тут машина, тут человек, тут дорога, здание, фон. Или медицинское изображение, снятое с микроскопа: нужно найти клетки, раковую опухоль и прочее. Разберёмся, как к такой постановке подступиться, исходя из тех инструментов, которые уже есть.

Начнём со сравнения. В классификации изображений всё было проще. На вход подаётся картинка

$$x \in \mathbb{R}^{H \times W \times C}$$

— высота, ширина,  $C$  цветовых каналов, — и на выходе нужно выбрать для неё одну метку класса

$$y \in \{1, \dots, K\}.$$

Как мы эту задачу решаем с помощью нейронных сетей? Берём картинку, подаём её в сеть — как правило, свёрточной архитектуры, — и на выходе получаем вектор оценок за классы: вектор длины  $K$ , по числу классов:

$$x \in \mathbb{R}^{H \times W \times C} \longrightarrow \boxed{\text{нейр. сеть } \theta} \longrightarrow f \in \mathbb{R}^K.$$

Дальше на этапе обучения подставляем этот вектор вместе с истинной меткой класса внутрь функции потерь — как правило, это кросс-энтропия:

$$\sum_{(x,y) \sim \text{Data}} -\log \text{softmax}(f(x, \theta))_y \longrightarrow \min_{\theta}.$$

В плане архитектуры это обычно свёрточная сеть с набором уменьшающихся по пространству слоёв — всё меньше и меньше, — а в конце один полносвязный слой, который переводит представление в нужное число классов; ставим softmax и решаем задачу.

## 2. Постановка задачи семантической сегментации

Теперь рассмотрим другую задачу — семантическую сегментацию изображения. Что значит «семантическую»? Можно рассматривать и просто сегментацию изображения: например, выделение суперпикселей — маленьких фрагментов изображения похожего цвета. Это просто сегментация: никакого обучения и никаких классов, чисто геометрическая нарезка картинки на однородные кусочки. Классический алгоритм здесь — SLIC (simple linear iterative clustering): по сути, k-means, где признаки пикселя — его цвет плюс координаты  $(x, y)$ , центры кластеров инициализируются по регулярной сетке, и каждый пиксель сравнивается только с ближайшими центрами, а не со всеми, — поэтому алгоритм очень быстрый. Получаются компактные однородные кусочки, границы которых хорошо липнут к границам объектов. Зачем это было нужно? Этот приём стоит запомнить — он ещё сыграет свою роль: если работать не с пикселями, а с суперпикселями («мета-пикселями»), любая последующая модель имеет дело с тысячами элементов вместо миллионов — в частности, в графе соседства, который мы

построим ниже, узлов становится в сотни раз меньше, и все вычисления резко дешевеют.

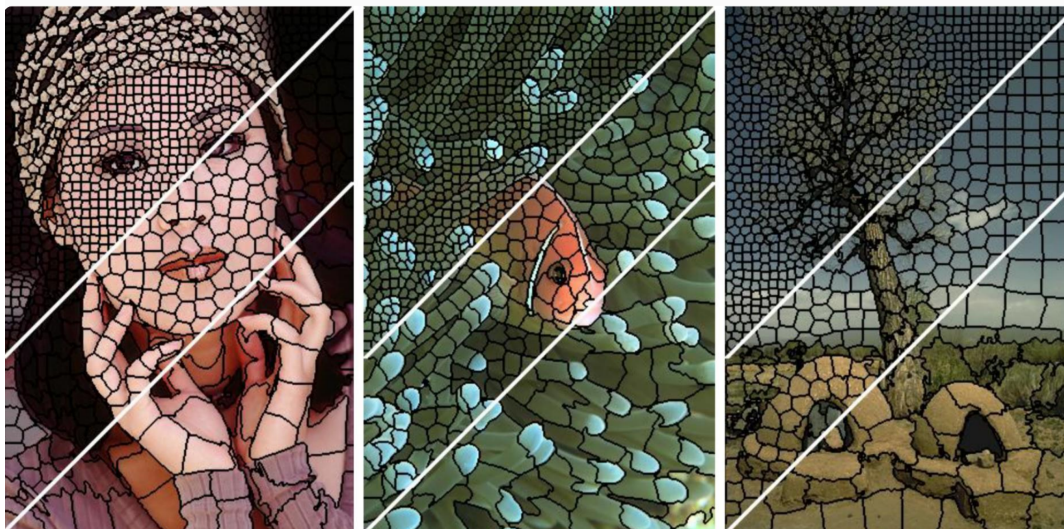


Рисунок 1. Суперпиксели SLIC на разной крупности нарезки: кусочки однородны, а их границы липнут к границам объектов (Achanta et al., 2012).

А семантическая сегментация — это когда сегменты имеют смысл: сегмент «человек», сегмент «машина», то есть смысловая разбивка изображения. Слова «класс» и «сегмент» здесь используются как синонимы.

В этой постановке на входе по-прежнему картинка  $x$ , а вот ответ  $y$  — это уже целая матрица: для каждого пикселя, по всей ширине и высоте, нужно выдать метку сегмента:

$$x \in \mathbb{R}^{H \times W \times C} \longrightarrow y \in \{1, \dots, K\}^{H \times W}.$$

Пространство ответов стало гораздо более богатым. И нам нужно подумать, что мы будем делать: на входе картинка, а на выходе, по сути, нужно уметь задавать вероятностное распределение в пространстве ответов при условии входа,  $p(y | x)$ . В задаче классификации это было распределение на конечном числе меток. Здесь же пространство ответов — это пространство сегментаций всех пикселей сразу, и распределение нужно задать в таком пространстве.

### 3. Энергетические модели и случайные поля

В общем виде такое распределение можно задавать конструкцией, которая под разными именами встречается в разных областях: *энергетические модели*, *марковские случайные поля*, *условные случайные поля* (CRF) — это всё примерно одно и то же; в статистической физике это распределение Гиббса. Задаётся некая функция энергии  $E(x, y, \theta)$ , определённая для входного изображения  $x$ , конкретной сегментации  $y$  и параметров модели  $\theta$ , и распределение имеет вид

$$p(y | x, \theta) = \frac{\exp(-E(x, y, \theta))}{Z(x, \theta)}, \quad Z(x, \theta) = \sum_{y'} \exp(-E(x, y', \theta)),$$

где суммирование в нормировочной константе  $Z$  идёт по всем возможным сегментациям  $y'$ . Чем меньше энергия какой-то конкретной конфигурации, тем более вероятна эта конфигурация согласно нашему распределению.

Отдельного внимания заслуживает нормировочная константа: в ней стоит сумма по всем возможным сегментациям. Сколько в ней слагаемых? В каждом из  $HW$  пикселей можно поставить одно из  $K$  значений, то есть слагаемых  $K^{HW}$  — экспоненциально большое множество. В этом главная техническая сложность: на практике мы не можем вычислить эту сумму на компьютере явно, это слишком дорого — тем более делать это на каждой итерации обучения.

Что за энергия здесь стоит? Её обычно задают в виде суммы двух групп слагаемых, которые называют *потенциалами*:

$$E(x, y, \theta) = \underbrace{\sum_i \psi_i(y_i; x, \theta)}_{\text{унарные потенциалы}} + \underbrace{\sum_{(i,j) \in \mathcal{E}} \psi_{ij}(y_i, y_j; x, \theta)}_{\text{парные потенциалы}}.$$

Первая сумма идёт по пикселям: слагаемые зависят индивидуально от метки данного пикселя — это *унарные потенциалы*. Вторая сумма идёт по парам пикселей из некоторого множества рёбер  $\mathcal{E}$  — это *парные потенциалы*. Какие именно пары пикселей входят в  $\mathcal{E}$ , задаётся *системой соседства* — о ней чуть ниже.

Что может стоять в качестве унарного потенциала? Нам нужно оценить «стоимость» назначения сегмента  $y_i$  пикселю  $i$  при входе  $x$ . По аналогии с задачей классификации возьмём минус логарифм softmax:

$$\psi_i(y_i; x, \theta) = -\log \text{softmax}(f_i(x, \theta))_{y_i}.$$

То есть теперь нейронная сеть строится так: на входе изображение  $x$ , а на выходе — объект такого же пространственного размера, как вход; сеть выдаёт не один вектор вероятностей, а свой вектор вероятностей  $f_i$  для каждого пикселя, по числу пикселей. Для истинной метки класса берём минус логарифм соответствующей вероятности.

Что такое  $\psi_{ij}$ ? Это потенциал, который определяется для пары пикселей. Его тоже можно вводить по-разному; самое каноническое введение такое: ставим индикатор того, что метки различаются, и умножаем его на экспоненту от разности тех же самых признаков  $f$ :

$$\psi_{ij}(y_i, y_j; x, \theta) = 1[y_i \neq y_j] \cdot \exp\left(-\frac{\beta}{2} \|f_i(x, \theta) - f_j(x, \theta)\|^2\right).$$

Что здесь записано? Мы хотим минимизировать энергию, когда конфигурация  $y$  хорошо соответствует изображению. По сути, мы штрафует такие ситуации: два пикселя мы хотим отнести к разным классам, при этом их признаки близки друг к другу. Самый простой пример: представьте, что два пикселя очень похожего цвета, а мы вдруг решили отнести их к разным сегментам, — за это накладывается штраф, мы хотим поменьше таких ситуаций. Или, допустим, если в качестве признаков пикселя взять просто его координаты: два пикселя близки друг к другу в пространстве, а мы отнесли их к разным сегментам — таких ситуаций тоже должно быть поменьше, тоже вносим штраф. Комбинируя унарный потенциал и парный потенциал, мы задаём наше распределение.

Как мы будем эту модель учить? Абсолютно так же, как в задаче классификации: по данным минимизируем минус логарифм правдоподобия,

$$-\log p(\bar{y} | x, \theta) \rightarrow \min_{\theta},$$

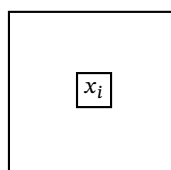
где  $\bar{y}$  — истинная сегментация из обучающей выборки.

## 4. Зачем нужны парные потенциалы: немного истории

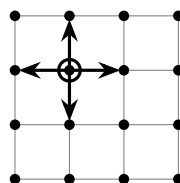
Резонный вопрос: почему вообще так делают — почему не ограничиться унарными потенциалами, зачем это дополнительное парное слагаемое? Это было сделано по историческим причинам. В досетевую эпоху, когда нейронных сетей ещё не было, задача сегментации изображения решалась обычно так: мы брали пиксель изображения и его небольшую окрестность и по этой окрестности каким-нибудь классификатором оценивали вероятность каждого класса.

Такие классификаторы были *слабыми*. Оно и понятно: из маленькой окрестности много закономерностей не извлечь. Если в ней в основном зелёные пиксели — наверное, класс «трава» должен иметь высокую вероятность; в основном голубые — наверное, небо или вода. Но более сложные ситуации так не разобрать: городская сцена, сумерки, всё серое, тёмное, коричневое, а границу нужно провести между человеком, тенью от машины и асфальтовой дорогой. Глядя на такую локальную информацию, а не на всю картину в целом, качественный классификатор не построишь.

И поэтому, когда модель сегментации строили на таких слабых классификаторах, простой выбор метки каждого пикселя по максимуму вероятности давал плохую сегментацию — «рванные» зоны, шум. Пришлось добавлять парные слагаемые, которые можно интерпретировать как некий *регуляризатор*. Что это за регуляризатор в простейшем случае, когда внутри парного потенциала даже нет признаков — просто  $\psi_{ij} = \mathbb{1}[y_i \neq y_j]$  по некоторой системе соседства? Система соседства — это структура графа на пикселях; самая простая — четырёхсвязная: соседи пикселя — те, что сверху, снизу, слева и справа:



изображение



решётка соседства (4-связность)

Тогда парный потенциал говорит просто: соседние пиксели в основном должны относиться к одному сегменту, а если мы вдруг отнесли их к разным — это стоит какой-то штраф. Слабый локальный классификатор, объединённый с таким регуляризатором, в сумме уже давал более-менее приличное качество.

Дальше эта конструкция развивалась: унарные потенциалы усиливались — сюда добавили нейронную сеть; парные потенциалы тоже можно усиливать — в них тоже вписаны признаки  $f$ , которые сеть может выучивать из данных, а не задавать вручную, плюс коэффициенты, и всё это мы будем обучать.

## 5. Обучение: градиент лог-правдоподобия

Теперь — что можно сделать с обучением. Обучаемся, как и раньше, минимизируя минус логарифм правдоподобия по всей выборке:

$$L(\theta) = \sum_{(x, \bar{y}) \sim \text{Data}} -\log p(\bar{y} | x, \theta) \rightarrow \min_{\theta}.$$

Подставим в минус логарифм правдоподобия наше распределение:

$$-\log p(\bar{y} | x, \theta) = E(x, \bar{y}, \theta) + \log Z(x, \theta).$$

Оптимизацию, как обычно, будем вести каким-то градиентным методом, то есть нам нужен градиент этого функционала по параметрам:

$$\nabla_{\theta}(-\log p(\bar{y} | x, \theta)) = \nabla_{\theta}E(x, \bar{y}, \theta) + \nabla_{\theta} \log Z(x, \theta).$$

С первым слагаемым проблем нет: в энергии стоят softmax, экспоненты — всё это легко дифференцируется обратным проходом по сети. Вопрос во втором слагаемом. Дифференцируем логарифм:

$$\nabla_{\theta} \log Z = \frac{\nabla_{\theta} Z}{Z} = \frac{1}{Z} \sum_y \nabla_{\theta} \exp(-E(x, y, \theta)) = \frac{1}{Z} \sum_y \exp(-E(x, y, \theta)) \cdot (-\nabla_{\theta} E(x, y, \theta)).$$

Градиент можно внести внутрь суммы — это очевидно; производная экспоненты даёт минус единицу на градиент энергии. Пока ничего хитроумного не произошло. Но теперь заметим:  $\exp(-E)/Z$  — это просто сама наша плотность  $p(y | x, \theta)$ . Получаем

$$\nabla_{\theta} \log Z = - \sum_y p(y | x, \theta) \nabla_{\theta} E(x, y, \theta) = - \mathbb{E}_{y \sim p(\cdot | x, \theta)} [\nabla_{\theta} E(x, y, \theta)].$$

Сумма по  $y$  вероятности на что-то — это просто математическое ожидание по этому распределению. Итого:

$$\nabla_{\theta}(-\log p(\bar{y} | x, \theta)) = \nabla_{\theta}E(x, \bar{y}, \theta) - \mathbb{E}_{y \sim p(\cdot | x, \theta)} [\nabla_{\theta} E(x, y, \theta)].$$

Градиент энергии для любой конкретной конфигурации мы считаем легко. Дальше вопрос: как посчитать математическое ожидание по распределению, у которого мы даже значение плотности точно вычислить не можем — из-за нормировочной константы?

Возможный выход такой: воспользуемся методом Монте-Карло. Предположим, что у нас есть какая-то процедура, которая умеет генерировать выборку из этого распределения:

$$y^{(1)}, \dots, y^{(M)} \sim p(y | x, \theta)$$

— индекс сверху означает номер сгенерированной конфигурации, а индекс снизу, как раньше, будет обозначать конкретный пиксель. Тогда математическое ожидание градиента энергии оценивается простым усреднением:

$$\mathbb{E}_y [\nabla_{\theta} E(x, y, \theta)] \approx \frac{1}{M} \sum_{m=1}^M \nabla_{\theta} E(x, y^{(m)}, \theta).$$

Как обычно поступают на практике. В нашем функционале стоит сумма по данным; в градиенте — градиент энергии для истинной сегментации минус матожидание. Получаются две стохастики: одна по данным — берём батч, — а для каждого выбранного  $x$  из батча генерируем вторую стохастическую, вот эти  $y$ . Если для одного конкретного объекта  $x$  смотреть, самое простое, что можно сделать, — брать

один сэмпл  $\hat{y}$ :

$$\nabla_{\theta}(-\log p(\bar{y} | x, \theta)) \approx \nabla_{\theta}E(x, \bar{y}, \theta) - \nabla_{\theta}E(x, \hat{y}, \theta), \quad \hat{y} \sim p(y | x, \theta).$$

Это стохастическая оценка градиента, причём несмещённая — её можно подставлять внутрь любой процедуры стохастического обучения.

Содержательно, если условно приравнять градиент к нулю, понятно, что происходит: мы фактически выравниваем градиентные поля для истинных, наблюдаемых сегментаций и для тех сегментаций, которые получаются, если сэмплировать из модельного распределения. Мы должны настроить потенциалы так, чтобы сегментации, вероятные по модели для данного входа, соответствовали истинным.

## 6. Сэмплирование по Гиббсу

Остался последний вопрос по существу: как же генерировать выборку из этого распределения? Что нам дано: некая плотность, которую мы можем вычислять для любой пары  $(x, y)$ , но с точностью до нормировочной константы — саму константу мы не знаем, можем посчитать только значение энергии. И нужно организовать процедуру генерации выборки.

Эту задачу можно сравнить с задачей численного интегрирования. Пусть нужно посчитать определённый интеграл от какой-то функции, значение которой можно вычислить в любой точке. Если интеграл одномерный, задача решается легко: формулы прямоугольников, трапеций — неважно; функция приближается гистограммой, и сумма считается. То же с генерацией: одномерная плотность всегда равномерно приближается гистограммой, а генерировать из гистограммы — это просто дискретное распределение, где высота столбика пропорциональна вероятности попасть в кусочек. В одномерном случае всё легко. Но в многомерном пространстве оценить интеграл размерности сто тысяч численно уже не выйдет: проклятие размерности, столбиками всё не замостить. То же самое происходит и с процедурой сэмплирования: как генерировать выборку в многомерном пространстве, имея доступ к значению плотности (с точностью до константы) в каждой точке?

Мы разберём один конкретный способ, чтобы перед глазами была хотя бы одна процедура, — *сэмплирование по Гиббсу* (Gibbs sampling). Как это работает. Мы хотим генерировать выборку из распределения  $p(y | x, \theta)$  и устраиваем некий итерационный процесс. Пусть в какой-то момент времени у нас есть  $y^{\text{old}}$  — старый вектор сегментации. Следующий вектор генерируем покомпонентно:

$$\begin{aligned} y_1^{\text{new}} &\sim p(y_1 | y_2^{\text{old}}, y_3^{\text{old}}, \dots, y_{HW}^{\text{old}}, x, \theta), \\ y_2^{\text{new}} &\sim p(y_2 | y_1^{\text{new}}, y_3^{\text{old}}, \dots, y_{HW}^{\text{old}}, x, \theta), \\ &\vdots \\ y_{HW}^{\text{new}} &\sim p(y_{HW} | y_1^{\text{new}}, y_2^{\text{new}}, \dots, y_{HW-1}^{\text{new}}, x, \theta). \end{aligned}$$

То есть исходно есть многомерное распределение на всём пространстве пикселей, а рассматривается одномерное условное распределение на первую переменную при условии всех остальных — а из одномерного распределения, как мы уже выяснили, генерировать легко. Затем вторая компонента генерируется из условного распределения при условии всех остальных, причём  $y_1$  подставляется

уже новый, только что сгенерированный. И так далее; для самого последнего пикселя уже будут все новые значения. Так получается новый вектор  $y$ ; затем итерации продолжаются, и следующий вектор генерируется по той же схеме. Можно доказать, что, пройдя сколько-то итераций этой процедуры, мы действительно будем получать выборку из нашего многомерного распределения.

Здесь можно провести аналогию с покоординатной оптимизацией: многомерная функция оптимизируется по одной переменной при зафиксированных остальных, потом по следующей, и так далее; при определённых ограничениях на функцию можно доказать, что такая оптимизация сходится. Тут то же самое, только для процедуры генерации.

Выпишем это одномерное условное распределение — что это за распределение  $p(y_i | y_{-i}, x, \theta)$ , где  $y_{-i}$  — все компоненты, кроме  $i$ -й. По формуле условной вероятности это совместная вероятность, делённая на вероятность условия, то есть с точностью до нормировки — чтобы сумма по всем значениям  $y_i$  давала единицу. А что такое наша совместная плотность? Экспонента от суммы слагаемых-потенциалов — то есть произведение экспонент от отдельных потенциалов. В этой экспоненте нас интересует только то, что зависит от  $y_i$ : всё, что от него не зависит, — константы, которые просто учтутся в нормировке. Поэтому достаточно записать

$$p(y_i = k | y_{-i}, x, \theta) = \frac{\exp(-\psi_i(k) - \sum_{j \in N(i)} \psi_{ij}(k, y_j))}{\sum_{k'=1}^K \exp(-\psi_i(k') - \sum_{j \in N(i)} \psi_{ij}(k', y_j))},$$

где  $N(i)$  — соседи  $i$ -го пикселя по системе соседства: больше ничего от метки этого пикселя не зависит. В числителе — выражение, где в качестве  $y_i$  подставлена метка  $k$ , а в знаменателе то же выражение просуммировано по всем меткам: мы просто нормируем. Такая формула легко вычисляется поэлементно — значит, к этому одномерному распределению у нас есть доступ, и процедура Гиббса запускается; она и используется в обучении. Существенно, что мы не вводили никаких дополнительных ограничений на структуру или форму потенциалов — здесь могут стоять любые функции.

Ещё один момент. Схеме Гиббса — и другим методам сэмплирования, которые устроены подобным образом (это методы Монте-Карло по схеме марковских цепей), — требуется некий этап *разогрева*. Обычно самый первый  $y$  берётся произвольно, например из случайной инициализации, и в начале итераций схема ещё не вышла в режим генерации выборки именно из целевого распределения — нужно сделать сколько-то итераций «вхолостую». Но вот как эта схема используется в обучении: на текущей итерации оптимизации для текущего  $\theta$  мы, допустим, уже считаем, что схема разогрелась, и умеем сэмплировать  $y$ . Потом мы сделали шаг метода оптимизации, то есть немного изменили параметры, — и на следующей итерации распределение для изменённого  $\theta$  будет похоже на предыдущее. Поэтому мы можем просто взять  $y$  с предыдущей итерации в качестве стартовой точки: от неё схема разогреется очень быстро, достаточно небольшого числа итераций — 5–10, — и мы уже выйдем на генерацию для нового  $\theta$ . Таким образом, процедура обучения у нас полностью построена.

## 7. Этап теста

Осталось понять, что происходит на этапе теста. Мы уже обучили параметры модели  $\theta$ ; для нового объекта  $x$  нужно понять, какую конфигурацию  $y$  выдавать. Разумно взять

$$\hat{y} = \arg \max_y p(y | x, \theta)$$

— сегментацию, в наибольшей степени соответствующую заданному тестовому объекту. Максимум плотности  $e^{-E}/Z$  — это то же самое, что минимум энергии: нужно просто минимизировать  $E(x, y, \theta)$  по  $y$ . Вообще говоря, это всё равно довольно сложная задача дискретной оптимизации: переменные дискретны, и минимизировать нужно по всем возможным сегментациям с учётом парных слагаемых. Для неё применяют разные алгоритмы; в зависимости от вида парных потенциалов  $\psi_{ij}$  для частных случаев можно вывести эффективные схемы. Но мы запишем общую схему — в той же логике рассуждений, через сэмплирование по Гиббсу, которым уже умеем пользоваться.

С помощью схемы Гиббса насэмплируем конфигураций  $y^{(1)}, \dots, y^{(M)}$  и оценим *маргиналы* — распределения на метку отдельного пикселя при условии входа:

$$p(y_i = k | x, \theta) \approx \frac{1}{M} \sum_{m=1}^M \mathbb{1}[y_i^{(m)} = k].$$

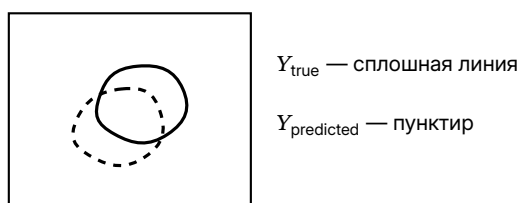
То есть мы считаем: в скольких из сгенерированных конфигураций на этом месте, для этого пикселя, встретилась конкретная метка класса, — и усредняем. После того как маргиналы оценены, ответ для каждого пикселя — просто максимум оценённого маргинала:

$$\hat{y}_i = \arg \max_k p(y_i = k | x, \theta).$$

В вычислительном плане это процедура такой же сложности, что и на каждой итерации обучения.

## 8. Метрика качества: коэффициент Жаккара

Прежде чем идти дальше — про величины, которые измеряются на этапе теста. В задаче классификации мы минимизируем кросс-энтропию на обучении, а в качестве измеряемой величины обычно берём точность распознавания (ассурагу). Каков аналог точности в задаче сегментации изображений? На обучении мы по-прежнему минимизируем функцию потерь — минус логарифмы вероятностей, к которым пришло сэмплирование по Гиббсу, — а измеряют обычно следующее. Самое простое — *усреднённый коэффициент Жаккара*. У нас есть картинка; для какого-то конкретного класса есть истинный сегмент  $Y_{\text{true}}$  и сегмент  $Y_{\text{predicted}}$ , который выдал наш алгоритм, — вычисленный, возможно, не очень точно:



Тогда можно померить площадь пересечения относительно площади объединения:

$$J = \frac{|Y_{\text{true}} \cap Y_{\text{predicted}}|}{|Y_{\text{true}} \cup Y_{\text{predicted}}|}.$$

Если сегментов много, делаем так для всех сегментов независимо и усредняем:

$$\bar{J} = \frac{1}{K} \sum_{k=1}^K \frac{|Y_{\text{true}}^k \cap Y_{\text{predicted}}^k|}{|Y_{\text{true}}^k \cup Y_{\text{predicted}}^k|}.$$

Средний коэффициент Жаккара по всем сегментам — это и есть аналог точности для задачи сегментации. (В англоязычной литературе то же самое называется IoU, intersection over union, а среднее — mean IoU.)

Хороший вопрос для самопроверки: а почему делим именно на площадь *объединения*? Вот что происходит с альтернативами. Если делить пересечение на площадь истинного сегмента, метрика максимизируется любым предсказанием, которое накрывает истину целиком, — например, «весь кадр — это машина»: пересечение равно истине, штрафа за лишнее нет. Если делить на площадь предсказанного сегмента, выгодна обратная стратегия: предсказать один-единственный пиксель, гарантированно лежащий внутри истины, — пересечение равно предсказанию, штрафа за пропущенное нет. И только деление на объединение наказывает одновременно и за лишние пиксели, и за пропущенные: увильнуть не получается ни в ту, ни в другую сторону.

Рядом с Жаккаром часто встречается и родственный коэффициент *Dice*:

$$D = \frac{2 |Y_{\text{true}} \cap Y_{\text{predicted}}|}{|Y_{\text{true}}| + |Y_{\text{predicted}}|}.$$

Он монотонно связан с Жаккаром —  $D = 2J/(1 + J)$ , — так что ранжируют модели обе метрики одинаково. Но у *Dice* есть отдельная практическая роль: если в формуле заменить жёсткие индикаторы принадлежности пикселя сегменту на предсказанные вероятности, получится гладкая, дифференцируемая функция (soft Dice) — и её можно поставить прямо в функцию потерь и учить сеть, максимизируя перекрытие непосредственно, обычно в комбинации с кросс-энтропией. С Жаккаром такой трюк тоже возможен, но исторически в задачах сегментации — особенно медицинских — прижился именно Dice loss.

## 9. Частный случай: только унарные потенциалы

И ещё одно замечание, прежде чем перейдём к архитектурам. Если бы парных потенциалов  $\psi_{ij}$  вообще не было — есть только унарные, — то наше многомерное распределение на  $u$  превратилось бы в независимое распределение по пикселям: нет ни одного сомножителя, который связывал бы метки разных пикселей. А для независимого распределения все процедуры упрощаются. Чтобы найти наилучшую конфигурацию как минимум энергии, можно просто независимо минимизировать унарный потенциал для каждого пикселя. Чтобы генерировать выборку, можно независимо сэмплировать из одномерных распределений каждого пикселя. Схема Гиббса, которая учитывает взаимосвязи между соседями, по сути перестаёт быть нужной.

Учёт парных потенциалов — это всегда хорошо с точки зрения качества сегментатора: если вся эта

процедура поднята, качество сегментации улучшается, иногда сильно. Но это даётся не бесплатно: на каждой итерации оптимизации нужно посчитать не просто градиент энергии обратным проходом по нейронной сети, а ещё дополнительно прогнать сэмплы — схему Гиббса; это дополнительные вычисления, и такая модель учится дольше, на практике, может быть, в несколько раз дольше. Поэтому всё зависит от сложности той задачи сегментации, которая решается. Если задача не слишком сложная и нейронная сеть, задающая унарные потенциалы, уже сама по себе справляется с сегментацией — можно просто отказаться от парных потенциалов, и всё. Но так бывает не всегда: бывают сложные изображения, и для них парные потенциалы приходится вводить, чтобы решать задачу качественно.

## 10. Данные для сегментации

Теперь, по аналогии с обзором свёрточных сетей, — популярные архитектурные решения — как строить нейронные сети для задачи сегментации. Таких сетей существует очень много, каждый год выходит что-то новое. Поэтому при встрече с задачей сегментации на практике лучше найти соответствующую свежую статью, посмотреть, какая архитектура хорошо себя показала на открытых данных, и использовать её. Здесь же разберём архитектурные идеи, которые встречаются во многих сетях, обладают общностью и используются не только для сегментации.

Сначала — немного контекста. Задача сегментации как стандартный бенчмарк оформилась давно: выборка Pascal VOC появилась в 2007 году, а первые подходы на основе свёрточных сетей — в 2014-м. Вот основные открытые выборки (объём обучающей и тестовой части и число классов):

| Выборка         | Обучение | Тест   | Классы |
|-----------------|----------|--------|--------|
| CamVid          | 468      | 233    | 11     |
| Pascal VOC 2012 | 9 963    | 1 447  | 20     |
| NYUDv2          | 795      | 645    | 40     |
| Cityscapes      | 2 975    | 500    | 19     |
| SUN RGB-D       | 10 355   | 2 860  | 37     |
| MS COCO '15     | 80 000   | 40 000 | 80     |
| ADE20K          | 20 210   | 2 000  | 150    |

(Плюс Mapillary Vistas, Apollo, соревнование Lyft, а из спутниковых — SpaceNet, xView и другие.)

Примечательно: основные выборки, на которых тестируются модели, по объёму — это единицы-десятки тысяч изображений. Это намного меньше, чем в классификации изображений, где счёт шёл на миллионы — ImageNet, наборы по триста миллионов картинок. Это не случайно: каждый объект для сегментации намного тяжелее собирать. Человек должен для заданного изображения разметить все пиксели — к какому классу они относятся, включая то, что происходит по границам объектов, — и трудоёмкость сбора такой разметки намного выше. Поэтому объёмы существующих выборок меньше, чем аналогичные для классификации.

Но есть и обратная сторона: раз выборка меньше, внутри каждого объекта намного больше информации, чем в задаче классификации. Там была одна метка на целую картинку, а тут — разметка

по классам для каждого пикселя; по сути, число объектов нужно умножить на размерность картинки, чтобы по объёму сигнала сравнивать с классификацией. Так что данных для обучения здесь на самом деле довольно много.

Плюс, как обычно, не забывайте про аугментации, которые на практике всегда нужно применять. Здесь есть тонкость: недостаточно просто слегка повернуть картинку или вырезать из неё кроп — аналогичное преобразование нужно синхронно делать и с маской сегментации: повернули картинку — поверните маску, вырезали кроп — вырежьте тот же кроп из маски. Синхронная аугментация входа и разметки — обязательная вещь.

Примеры из жизни, где задача сегментации используется: медицинские изображения; спутниковые снимки — вплоть до детектирования тюленей, карты городов, дороги и дома, наблюдение за изменением площадей посевов и береговых линий; выделение объекта от фона; и, наверное, самый важный пример — беспилотные автомобили: трёхмерная сегментация по данным лидара — машина замеряет пространство вокруг, и нужно сегментировать человека, светофор, другую машину, причём делать это ещё и быстро.

## 11. Архитектуры: SegNet, U-Net, LinkNet

Какие же архитектуры на основе нейросетей можно применять для этой задачи?

Небольшая предыстория. Самая первая мысль, с которой всё началось, — *полностью свёрточная сеть* (FCN, fully convolutional network, 2014–2015 годы): раз мы реализуем отображение «тензор в тензор», картинка в карту меток, можно просто выбросить из классификационной сети полносвязные слои и оставить одни свёртки — тогда сеть сможет принимать вход любого размера и выдавать пространственный выход. Проблема в том, что делать свёртки в полном разрешении дорого, а после уменьшения разрешения ответ получается грубым; авторы FCN стали подмешивать к грубому выходу карты с более ранних, крупных слоёв (варианты FCN-32s, FCN-16s, FCN-8s — число показывает, во сколько раз итоговая карта грубее входа). По сути, это первый шаг в сторону тех самых skip connections, которые мы сейчас увидим в развитом виде.

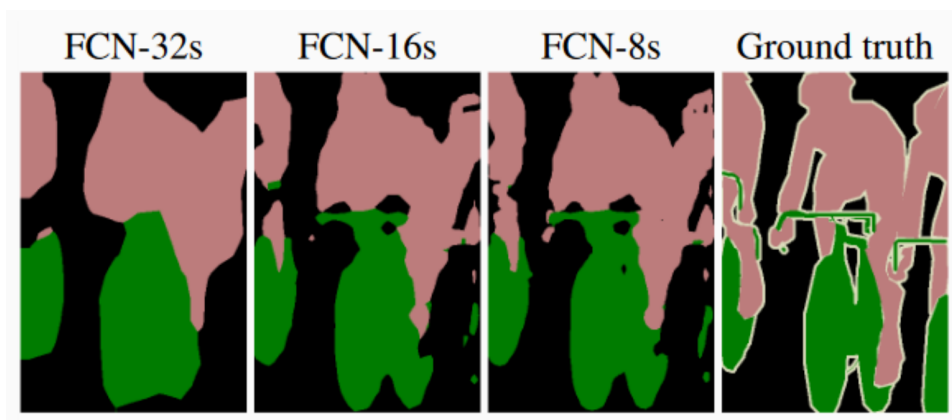


Рисунок 2. Чем больше карт с ранних слоёв подмешано (FCN-32s → FCN-8s), тем точнее границы (Long et al., 2015).

**SegNet.** Первая, можно сказать, базовая архитектура — то, что первым приходит в голову: модель типа

**кодировщик–декодировщик.** На вход — картинка; кодировщик — последовательность свёрточных слоёв, уменьшающих пространственное разрешение; где-то на внутреннем слое получаются высокоуровневые признаки — маленькие карты, которые хорошо описывают входное изображение. После чего из этого редуцированного пространства признаков мы такой же последовательностью свёрточных слоёв, только расширяющихся, выводим объект такого же размера, что и входное изображение, — по вектору вероятностей на каждый пиксель. Полносвязных слоёв в сети нет вообще, поэтому она получается довольно легковесной. Парные потенциалы здесь не используются — просто максимум по каждому пикселю.

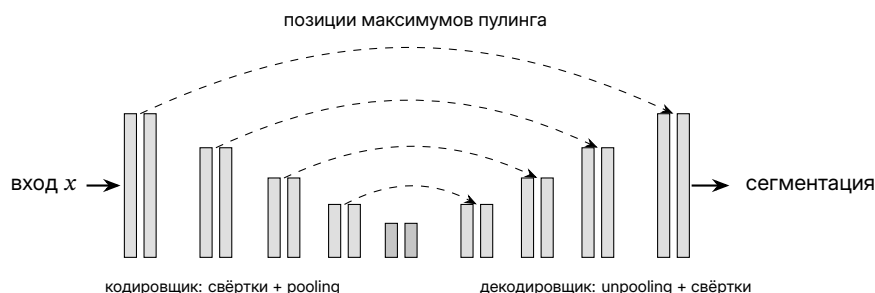


Рисунок 3. Архитектура SegNet: кодировщик–декодировщик; из каждого пулинга кодировщика в парный unpooling декодировщика передаются позиции максимумов.

Общая схема «кодировщик–декодировщик со skip connections», к которой мы сейчас придём, выглядит так:

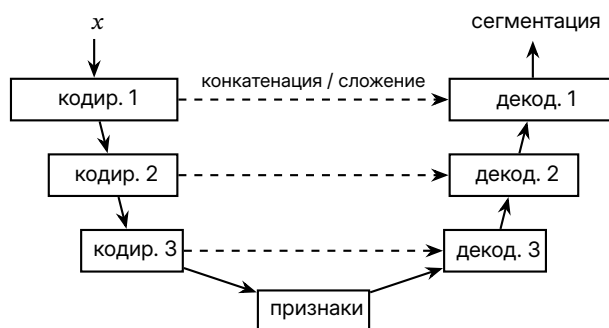


Рисунок 4. Схема «кодировщик–декодировщик со skip connections»

Пунктирные стрелки — те самые прямые связи между уровнями одинакового пространственного разрешения; в SegNet их роль играет только перенос позиций максимумов, в U-Net — конкатенация карт, в LinkNet — сложение.

Понятно, что это самое простое, что можно предложить, и тут же возникает проблема, на осознание и решение которой человечеству потребовалось некоторое время. В середине сети остаётся маленькое внутреннее представление — маленькая «картинка», сжатый набор признаков, — и из этого малого представления нужно вывести сегментацию в полном разрешении: попиксельную разметку, где граница проходит именно в этом пикселе, а не в соседнем. Точно восстановить все границы из усреднённого малого представления невозможно. Наглядная интерпретация: в глубине сети известно, что уши кота где-то слева, глаза где-то в середине, хвост где-то справа, — а нарисовать кота нужно попиксельно. Почти наверное получится не то: сеть помнит, что есть на картинке, но уже не помнит, где именно это было.

Авторы SegNet эту проблему осознали — хотя решение «не докрутили». Что они сделали: процедуры, увеличивающие вдвое размер очередного внутреннего представления (upsampling), они связали с пулингом кодировщика. На проходе вперёд, в слоях кодировщика, max-pooling  $2 \times 2$  уменьшает изображение вдвое, и мы запоминаем, с каких позиций брался максимум (это в любом случае нужно, чтобы прокидывать градиент назад). А на этапе декодировщика, когда мы увеличиваем картинку вдвое (unpooling), мы расставляем значения в те самые позиции, с которых max-pooling брал максимум при проходе вперёд. Происходит перенос пространственной информации: чтобы построить большое изображение, мы используем не только предыдущий слой декодировщика, но и кое-что со входа соответствующей размерности. К сожалению, эта идея оказалась слишком слабой: запоминать одни лишь позиции максимумов недостаточно — на самом деле нужно запоминать больше. (Из той же эпохи — DeconvNet: кодировщик из предобученной VGG16, а декодировщик — зеркальная башня из unpooling и транспонированных свёрток; идея та же, проблема та же.)

Для полноты картины — четыре стандартных способа увеличить разрешение карты вдвое; unpooling из SegNet — последний из них:

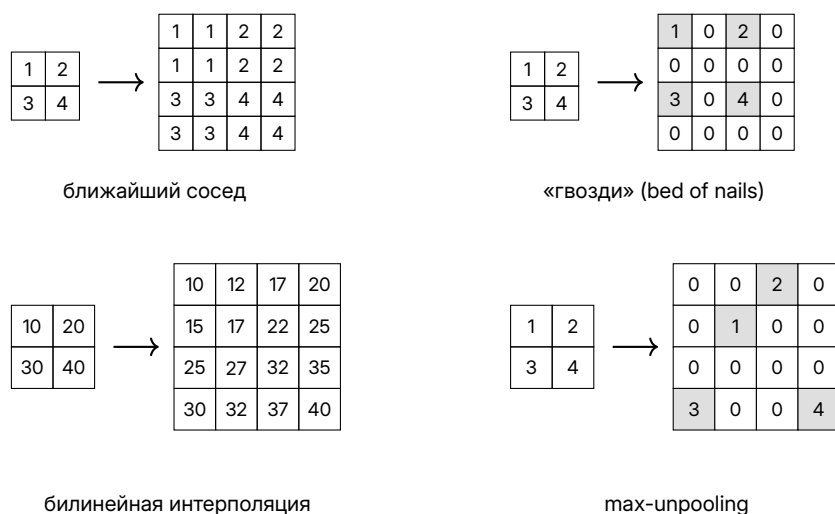


Рисунок 5. Способы увеличения  $2 \times 2 \rightarrow 4 \times 4$ : копирование ближайшего соседа; «гвозди» --- значение в один угол блока, остальное нули; билинейная интерполяция; max-unpooling --- значения расставляются в те позиции (подсвечены), с которых max-pooling брал максимум при проходе вперёд.

**U-Net.** Следующая идея, которая сработала, — сеть U-Net, ставшая классикой. Важно сказать, что U-Net изначально пришла из сообщества по обработке медицинских изображений, поэтому её архитектура может показаться странной на фоне того, как это принято делать в компьютерном зрении. Снова структура кодировщик-декодировщик: свёрточные слои уменьшают разрешение, потом увеличивают. Ключевой момент: когда мы увеличиваем разрешение в декодировщике, нужно использовать информацию со входа на соответствующем уровне разрешения. Что сделали авторы: они взяли *конкатенацию*. Есть выход очередного блока декодировщика; увеличиваем его размер вдвое и просто конкатенируем с соответствующим по разрешению куском кодировщика — фильтров-каналов становится вдвое больше, — дальше обрабатываем свёрточными слоями, и на следующем уровне опять конкатенируем с выходом соответствующего слоя кодировщика.

Ключевая идея: сеть не учится напрямую выводить сегментацию из маленького среднего слоя сразу в полном разрешении. Вместо этого на каждом шаге она понемногу уточняет границы: делаем upsampling, увеличиваем — и добавляем немного деталей, пользуясь информацией с соответствующего

слоя кодировщика; так, добавляя детали к границам, в конечном итоге выводим качественную попиксельную разметку. Идея пробрасывать прямые связи между кодировщиком и декодировщиком на соответствующих уровнях используется повсеместно — не только для сегментации изображений и не только для обработки текстов; та же идея используется в современных диффузионных генеративных моделях изображений: мы не генерируем изображение в полном разрешении сразу, а выводим его постепенно, через промежуточные представления всё увеличивающегося разрешения.

Любопытный исторический штрих: статья про U-Net вышла в медицинском журнале, и на популярных бенчмарках по сегментации авторы её даже не измеряли. Признание пришло из соревнований: U-Net оказалась частью множества решений-победителей — от Ultrasound Nerve Segmentation (2016) до Kaggle Data Science Bowl (2018), TGS Salt (2018) и Airbus Ship Detection (2018).

Откуда эта странность, видно по размерностям в оригинальной статье: вход  $572 \times 572$  (а не 224 или другая стандартная величина), дальше свёртки  $3 \times 3$  без *паддинга* —  $572 \rightarrow 570 \rightarrow 568$ , на каждом свёрточном слое картинка немного уменьшается; потом *max-pooling*  $2 \times 2$  уменьшает вдвое, и опять понемногу. Из-за отсутствия *паддинга* на этапе декодировщика происходит вот что: чтобы сконкатенировать выход декодировщика с куском кодировщика, нужен одинаковый пространственный размер, а он не совпадает — поэтому из карт кодировщика вырезается центральная часть (*crop*), и уже она конкатенируется. Авторы просто не пользовались привычными нам идеями — например, из ResNet, — что на одном уровне пространственную размерность сохраняют.

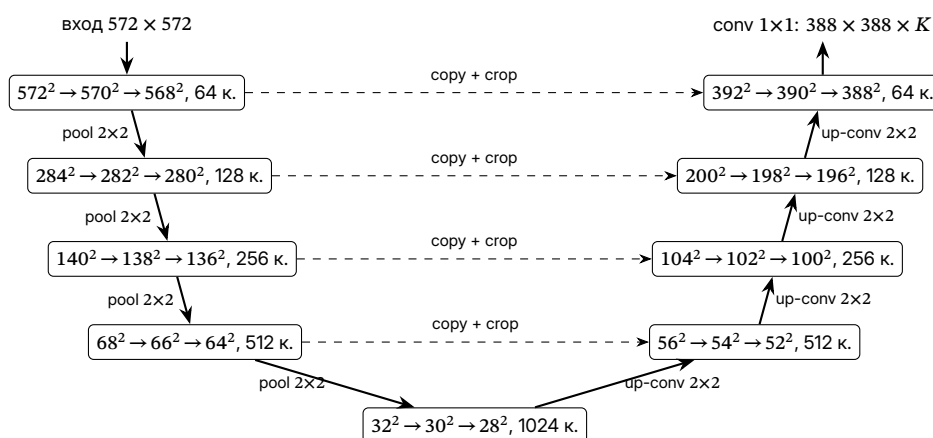


Рисунок 6. Схема U-Net с размерами из оригинальной статьи: на каждом уровне две свёртки  $3 \times 3$  без *паддинга* (те самые  $572 \rightarrow 570 \rightarrow 568$ ), пунктиром --- перенос карт кодировщика с обрезкой до размера декодировщика.

Зелёные стрелки на схеме — это увеличение размера изображения вдвое; делать это можно по-разному: мы обсуждали транспонированные свёртки, за счёт которых можно увеличивать размерность, а здесь используется очень популярная в изображениях процедура *upsampling* — просто интерполяция, увеличивающая разрешение вдвое. Сама архитектура оказалась удачной: идея «кодировщик–декодировщик с прокидыванием связей на соответствующих пространственных разрешениях» очень успешна и хорошо решает много задач.

Раз уж зашла речь про транспонированные свёртки, две ремарки. Первая: мы пользуемся этим названием, но ни разу не сказали, откуда оно. Свёртка — линейная операция, значит, её можно записать как умножение на матрицу. Возьмём одномерный пример: сигнал  $\mathbf{a} = (a, b, c, d)^T$ , ядро  $(x, y, z)$ , шаг 1, паддинг 1. Тогда свёртка — это

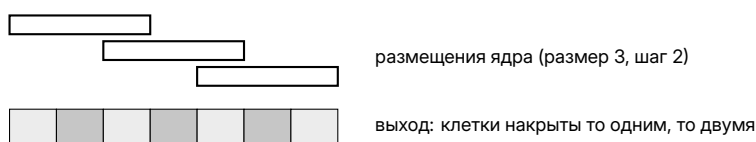
$$\mathbf{X} \mathbf{a}_{\text{pad}} = \begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & x & y & z & 0 & 0 \\ 0 & 0 & x & y & z & 0 \\ 0 & 0 & 0 & x & y & z \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ ax + by + cz \\ bx + cy + dz \\ cx + dy \end{bmatrix} :$$

каждая строка матрицы — сдвинутая копия ядра. А транспонированная свёртка определяется как умножение на *транспонированную* матрицу того же вида:

$$\mathbf{X}^T \mathbf{a} = \begin{bmatrix} x & 0 & 0 & 0 \\ y & x & 0 & 0 \\ z & y & x & 0 \\ 0 & z & y & x \\ 0 & 0 & z & y \\ 0 & 0 & 0 & z \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} ax \\ ay + bx \\ az + by + cx \\ bz + cy + dx \\ cz + dy \\ dz \end{bmatrix}$$

— операция, «обратная по размерностям»: свёртка с шагом  $s$  уменьшала длину сигнала, транспонированная к ней — увеличивает. Никакого обращения самой свёртки тут нет, только обращение формы, и все элементы ядра остаются обучаемыми — поэтому это *обучаемое* увеличение разрешения, в отличие от фиксированной интерполяции.

Вторая ремарка — предостережение: *эффект шахматной доски*. Когда транспонированная свёртка увеличивает разрешение, каждый выходной пиксель собирается из тех размещений ядра, которые его накрывают, — и если размер ядра не делится на шаг, разные пиксели выхода накрываются разным числом размещений:



Пиксели, куда попало больше размещений, систематически получают «ярче» соседей — на выходе проступает регулярная клетчатая рябь, особенно заметная, когда таких слоёв несколько подряд. Вот как это выглядит на сгенерированных изображениях, если присмотреться:

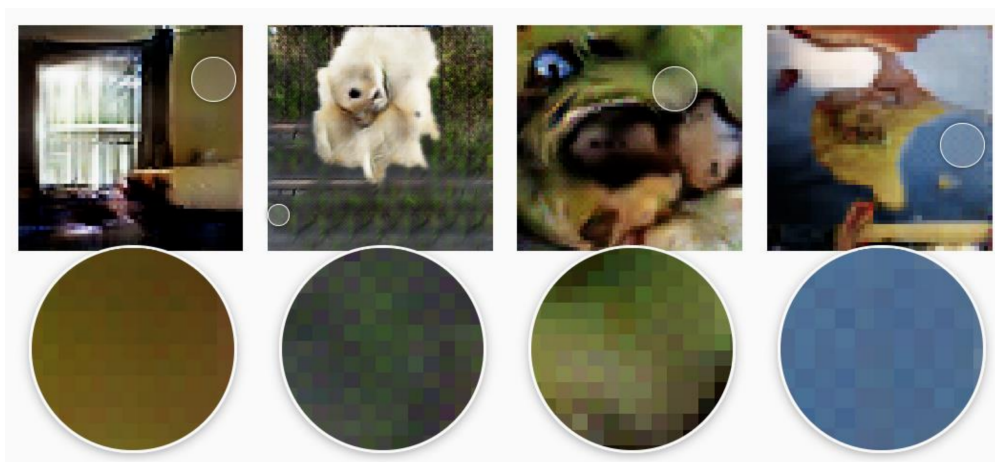


Рисунок 7. Эффект шахматной доски: под увеличением видна регулярная клетчатая структура (Odena et al., Distill, 2016).

Лечится это просто: либо берите размер ядра кратным шагу (например,  $4 \times 4$  при шаге 2), либо вообще замените обучаемое увеличение на пару «интерполяция + обычная свёртка» — поэтому в U-Net-подобных сетях так часто и стоит именно интерполяция.

**LinkNet.** Сюда же примыкает архитектура LinkNet, которая очень похожа на U-Net, поэтому имеет смысл упомянуть её сразу. Та же идея кодировщик–декодировщик с прокидыванием связей на соответствующих уровнях, но статья вышла на несколько лет позже: авторы попытались учесть недостатки предыдущих архитектур и сделать то же самое по качеству, но максимально легковесно — поменьше параметров, быстрее обучать. Одно из отличий: вместо конкатенации (после которой надо ещё свёрткой «перемешивать» удвоенные каналы) используется простое сложение соответствующих карт — это быстрее; получается аналог декодера, только с плюсом. И здесь уже честно используются все достижения свёрточных сетей для классификации: свёртки  $3 \times 3$  с паддингом, чтобы на одном уровне размерность не менялась; остаточные связи (skip connections) через каждые два свёрточных слоя; в декодировщике — идея уменьшения числа каналов по схеме bottleneck: сначала свёртка  $1 \times 1$ , уменьшающая число фильтров вчетверо, затем на этом редуцированном пространстве свёртка  $3 \times 3$  (так быстрее), затем  $1 \times 1$ , возвращающая исходное число каналов, — как это было, например, в Inception; и сложение вместо конкатенации, чтобы вычислений было меньше. Увеличение разрешения вдвое в декодировщике делается транспонированными свёртками, а свёртки  $1 \times 1$  сокращают число каналов вчетверо перед основной работой. В итоге по качеству LinkNet сопоставима с PSPNet (о ней ниже), а по времени работы — примерно вдвое быстрее аналога на ResNet-18; архитектура успешно использовалась в ряде соревнований.

**RefineNet.** Ещё один пример из того же семейства — RefineNet: та же многоуровневая схема со skip connections, но на этапе декодирования вместо простых свёрток стоят сложные «улучшайзеры» (enhancers): остаточные блоки в духе ResNet, блоки смешивания карт разных пространственных масштабов, агрегаторы контекста. Каждый уровень, кроме самого нижнего, конкатенирует входы с двух разных масштабов. Идеино это всё то же уточнение границ от уровня к уровню — просто каждый шаг уточнения сделан «пожирнее».

**Hourglass.** Идею «кодировщик–декодировщик» можно применять и многократно: сеть типа «песочных часов» (stacked hourglass) — это несколько модулей кодировщик–декодировщик, поставленных друг за другом: первый выдаёт грубую карту, второй берёт её вместе с признаками и уточняет, третий уточняет ещё раз, и так далее; после каждого модуля обычно ставится промежуточная функция потерь, чтобы каждая «рюмка» училась выдавать осмысленный результат. Архитектура пришла из

задачи оценки позы человека — про саму эту задачу и про то, почему её решают той же машинерией «изображение в изображение», можно почитать в приложении в конце конспекта, — но приём «повторное уточнение каскадом» общий и встречается много где.

**HRNet.** И совсем другой ответ на ту же проблему потери разрешения: а можно его вообще не терять. В HRNet (high-resolution network) ветка полного разрешения идёт через всю сеть от начала до конца; параллельно с ней постепенно заводятся ветки пониженных разрешений, и все ветки регулярно обмениваются информацией — карты пониженных разрешений интерполируются вверх и подмешиваются в ветку высокого разрешения, и наоборот. Не нужно ничего восстанавливать: точная пространственная информация никуда не пропадала. Платить за это приходится памятью и вычислениями — свёртки в полном разрешении дороги, с этого начиналась лекция, — но для задач, где границы критичны, такой обмен бывает выгодным.

Небольшое отступление — чтобы показать, что схема «кодировщик–декодировщик» применяется далеко за пределами сегментации. Возьмём семейство дифференциальных уравнений в частных производных: по дискретизациям коэффициентов уравнения и правой части на сетке нужно предсказать дискретизацию решения. Но это же снова задача «изображение в изображение» — и U-Net для неё подходит на удивление хорошо. Почему? Решения физически осмысленных уравнений — гладкие функции: полезной информации в них немного (несколько первых коэффициентов разложения — как «уши, глаза и хвост» в примере с котом), и её можно сжать кодировщиком, а затем декодировать в поле решения, подглядывая в skip connections за деталями. На этой идее выросло целое направление *нейронных операторов* (neural operators) — суррогатных моделей физики, где U-подобные архитектуры решают уравнения на порядки быстрее классических численных методов. Так что U-Net ещё встретится — и не раз, и не только в компьютерном зрении.

## 12. Пирамидальный пулинг и свёртки с дырками

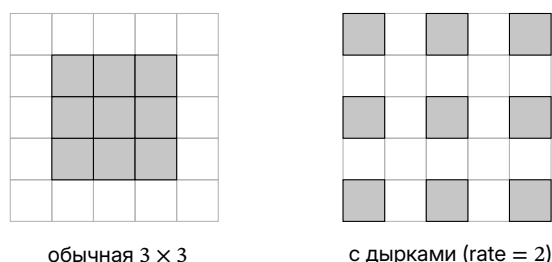
Это была первая базовая идея построения архитектур для сегментации — и не только для сегментации. Вторая идея — так называемый *пирамидальный пулинг* (spatial pyramid pooling).

О чём здесь речь. Мы решаем задачу сегментации, а на изображениях одни и те же объекты могут быть разного размера: люди на переднем плане покрупнее, на заднем — помельче; заранее масштаб каждого объекта мы не знаем. При этом во всех предыдущих архитектурах свёртки были, скажем,  $3 \times 3$  — они действуют очень локально, смотрят в маленькое окошко, и им сложно качественно сегментировать объекты разного масштаба.

Что предлагается: сделать агрегации по пространству на нескольких масштабах сразу. Берём трёхмерный выход какого-то слоя сети и применяем к нему несколько пулингов. Первый — глобальный: усредняем всю карту целиком, по каждому фильтру получаем одно число — как будто «окинули взглядом» всю картинку. Второй — режем карту на блоки  $2 \times 2$  и усредняем внутри каждого блока. Потом  $3 \times 3$ , ещё чуть подробнее, и так далее — пирамида агрегаций разной крупности. Дальше к каждому результату применяется свёртка  $1 \times 1$ , чтобы поправить число каналов, все эти карты интерполяцией увеличиваются до одинакового пространственного размера с исходной картой — и конкатенируются с ней. Детали здесь не так важны; важна идея: чтобы одновременно смотреть и

на большие объекты, и на маленькие, нужны глобальные усреднения по пространству на нескольких масштабах. Это и есть *spatial pyramid pooling*. Такая схема используется, например, в PSPNet (Pyramid Scene Parsing Network): там backbone — ResNet, у которого после нескольких первых слоёв все свёртки заменены на растянутые (о них ниже); к выходам блоков ResNet добавлены вспомогательные функции потерь; а модуль пирамидального пулинга ставится на выход сети и собирает глобальный контекст. Пирамида помогает избежать грубых семантических ошибок, но попиксельная детализация страдает: в конце карта увеличивается интерполяцией сразу вчетверо.

Вторая реализация той же идеи — *свёртки с дырками* (atrous, или dilated convolutions). Это свёртка  $3 \times 3$ , у которой ядро применяется «растянуто» по пространственным размерностям: между элементами ядра вставляются пропуски, так что рецептивное поле становится больше при том же числе параметров:



Заштрихованы клетки, на которые смотрят девять весов ядра: параметров по-прежнему девять, а окно — уже  $5 \times 5$ . (Оговорка: теоретическая сложность при этом растёт, а практическая — страдает ещё сильнее, так что бесплатным этот приём не назовёшь.) И снова строится пирамида — к одной и той же карте признаков применяется несколько таких свёрток со всё большим растяжением (например, с rate 6, 12, 18, 24); растянутые свёртки агрегируют информацию на больших пространственных масштабах; затем всё, как обычно, конкатенируется и посылается дальше. Это тот же пирамидальный пулинг, только через свёртки с дырками (схема ASPP — atrous spatial pyramid pooling).

Итого у нас две основные идеи: кодировщик–декодировщик со skip connections и пирамидальные пространственные агрегации — либо через пулинги, либо через свёртки с дырками. В хороших архитектурных решениях обычно используются обе идеи одновременно, плюс что-то ещё.

**DeepLab.** Пример — семейство моделей DeepLab (v2, v3, v3+; тоже от Google, как и Inception в классификации): модель развивалась с годами, и как только где-то находились хорошие архитектурные решения, их добавляли в общую модель. В какой-то момент DeepLab выглядел так: входное изображение; свёрточная сеть, последовательно уменьшающая размерность, но работающая не локальными свёртками  $3 \times 3$ , а на каждом уровне — пирамидальными агрегациями, чтобы действовать на разных масштабах. А затем сверху добавили CRF — то, что мы обсуждали в самом начале: парные потенциалы, причём полносвязные — каждый пиксель связан с каждым, все пары. Чтобы это быстро обчислять, потребовались специальные алгоритмические ухищрения — слагаемых очень много, — но это можно сделать; и через эту надстройку идёт градиент, примерно по той же схеме, что мы обсуждали с сэмплингом по Гиббсу. В поздних версиях (v3+ и далее) добавили и кодировщик–декодировщик: сначала кодировщик из многих уменьшающихся слоёв, потом декодировщик со skip connections с соответствующих разрешений кодировщика; в каких-то версиях используется CRF — всё вместе даёт качественное решение. Лучшая конфигурация на тот момент (предобучение на MS COCO, замер на Pascal VOC 2012) — ResNet-101 + свёртки с дырками + ASPP + CRF; работает отлично, но получается очень тяжёлой.

В базовом варианте таких моделей backbone — это предобученная сеть для классификации: берём входное изображение, ставим кусок предобученного ResNet — не до самого конца, не до  $7 \times 7$ , а до карт разрешения около  $32 \times 32$ , — фиксируем предобученные веса, и уже поверх этого ставим пирамидальные свёртки и всю остальную схему. То есть внутри стоит классическая CNN для классификации изображений.

Дальше есть много более сложных архитектур; при решении этой задачи разобраться в них несложно: когда базовые вещи понятны, дальше уже легко. Но для несложных задач сегментации изображений обычно хватает архитектур типа U-Net: достаточно взять LinkNet или U-Net, поставить и обучить.

## 13. Что дальше

Напоследок — пара слов о том, куда эта область растёт дальше. Эта лекция была про *семантическую* сегментацию: каждому пикселю — класс. Есть постановка посложнее — *instance-сегментация*: пиксели одного класса нужно ещё и разделить на отдельные объекты (не «вот пиксели класса „человек“», а «вот первый человек, вот второй»). Объединение обеих постановок называется *паноптической сегментацией*.



Рисунок 8. Слева semantic: все люди --- один класс «человек»; справа instance: каждый человек --- отдельный объект.

Классическое решение instance-задачи — Mask R-CNN: берётся детектор ограничивающих рамок (Faster R-CNN — про такие детекторы как раз следующая лекция), и в каждую найденную рамку добавляется маленькая полностью свёрточная голова, которая рисует маску объекта внутри рамки. Так что после следующей лекции этот метод станет понятен целиком. Современные же модели паноптической сегментации — Mask2Former, OneFormer — построены на *трансформерах*; к этому механизму мы придём в блоке про обработку текстов, а пока достаточно запомнить имена: одна и та же модель там решает семантическую, instance- и паноптическую постановки сразу.

Есть и ещё одна практичная постановка — *интерактивная* сегментация: пользователь кликает по объекту, модель дорисовывает маску, следующий клик её уточняет (типичный представитель — RITM); главный инструмент этой линии сегодня — модель Segment Anything (SAM): большая модель,

обученная выделять маску произвольного объекта по подсказке — клику или рамке, — которую сейчас часто используют как универсальный инструмент, в том числе для быстрой разметки данных. Со всем этим легко разобраться на базе материала этой лекции.

Следующая же тема — *детекция объектов* на изображении: теперь нужно не сегментировать, а найти и выделить на изображении ограничивающие рамки (bounding boxes) — где находятся объекты и что это за объекты. С точки зрения выхода это опять сложное структурное пространство: мы заранее не знаем, сколько на данном изображении объектов, какого они размера; они могут перекрываться, стоять один за другим. Поэтому методы решения там будут ещё сложнее, чем в сегментации изображений. Об этом — следующая лекция.

## Приложение: оценка позы человека

Задача, о которой шла речь в абзаце про Hourglass. *Оценка позы* (human pose estimation): по изображению человека нужно найти координаты его суставов — ключевых точек скелета (плечи, локти, запястья, колени и так далее). На первый взгляд это совсем не сегментация: выход — не карта меток, а набор точек.

Первый нейросетевой подход так задачу и решал. DeepPose (2014) — прямая регрессия: свёрточная сеть, в конце полносвязный слой, который выдаёт вектор из  $2N$  чисел — координаты  $N$  суставов. Работало, но не очень: точную координату регрессией предсказывать тяжело, а если сустав не виден или людей несколько, постановка вообще ломается.

Прорыв случился, когда задачу переформулировали в уже знакомых терминах: предсказывать не координаты, а *тепловые карты* (heatmaps) — по одной карте на сустав, где значение в пикселе — «уверенность», что сустав находится здесь. Истинная разметка — гауссова «горка» вокруг истинной позиции сустава. Но карта на каждый сустав — это же в точности выход сегментационной сети: вход-изображение, выход-изображение,  $N$  каналов вместо  $K$  классов! Поэтому здесь работает вся машинерия этой лекции: кодировщик-декодировщик, skip connections, каскадное уточнение — stacked hourglass как раз и была придумана для тепловых карт суставов. Финальный ответ — максимум по каждой карте.



Рисунок 9. Тепловые карты суставов: слева вход, дальше --- карта на каждый сустав; предсказание --- максимум карты (Toshev, Szegedy, 2014).

Дальше — многолюдные сцены. OpenPose (2017) научилась в реальном времени находить позы всех людей в кадре сразу: помимо тепловых карт суставов сеть предсказывает ещё и *поля родства частей* (part affinity fields) — векторные поля вдоль конечностей, по которым найденные суставы собираются

в скелеты конкретных людей (какой локоть к какому плечу относится). Обе карты — снова выходы типа «изображение в изображение».

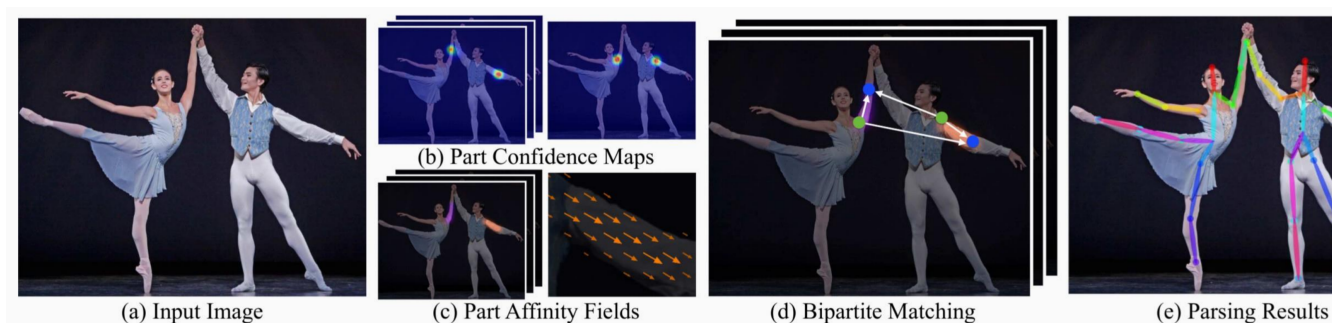


Рисунок 10. Конвейер OpenPose: тепловые карты суставов + поля сродства частей → сборка скелетов конкретных людей (Cao et al., 2017).

Вывод для нашего курса: «сегментационная» архитектура — это на самом деле универсальный инструмент для любых задач плотного предсказания по картинке, будь то метки классов, тепловые карты суставов, векторные поля или решения дифференциальных уравнений из отступления про нейронные операторы.

Почитать: [DeepPose — Toshev, Szegedy. Human Pose Estimation via Deep Neural Networks. CVPR 2014;](#)  
[Stacked Hourglass — Newell et al. ECCV 2016;](#) [OpenPose — Cao et al. Realtime Multi-Person 2D Pose Estimation using P](#)